

MATA54 - Estruturas de Dados e Algoritmos II

Busca de Padrões em Textos

Flávio Assis

Versão gerada a partir de slides do Prof. George Lima

v1.1

MATA54 - Estruturas de Dados e Algoritmos II

Busca de Padrões em Textos

Flávio Assis

Versão gerada a partir de slides do Prof. George Lima

v1.1

Motivação: Busca de Padrões em Textos

Aplicações: busca de uma subsequência de caracteres

- ▶ **Texto:** sequência de caracteres $T[1..n]$ de tamanho n
- ▶ **Alfabeto:** cada caractere do texto é um elemento de um conjunto (alfabeto) Σ
- ▶ **Padrão:** sequência de caracteres $P[1..m]$ de tamanho $m \leq n$
- ▶ Quais são as posições em T onde P ocorre (se ocorrer)?

Algoritmos a serem estudados:

- ▶ Algoritmo baseado em Autômato
- ▶ KMP (Knuth-Morris-Pratt)
- ▶ BM (Boyer-Moore)
- ▶ RK (Rabin-Karp)

Algoritmo Simples (Ingênuo): Ilustração

Ex: **T** = “ababababaababb” e **P** = “ababb”

1	2	3	4	5	6	7	8	9	10	11	12	13	14
a	b	a	b	a	b	a	b	a	a	b	a	b	b

Algoritmo Simples (Ingênuo): Ilustração

Ex: **T** = "ababababaababb" e **P** = "ababb"

1	2	3	4	5	6	7	8	9	10	11	12	13	14
a	b	a	b	a	b	a	b	a	a	b	a	b	b
a	b	a	b	b									

Algoritmo Simples (Ingênuo): Ilustração

Ex: **T** = "ababababababb" e **P** = "ababb"

1	2	3	4	5	6	7	8	9	10	11	12	13	14
a	b	a	b	a	b	a	b	a	a	b	a	b	b
a	b	a	b	b									
→	a	b	a	b	b								

Algoritmo Simples (Ingênuo): Ilustração

Ex: **T** = "ababababababb" e **P** = "ababb"

1	2	3	4	5	6	7	8	9	10	11	12	13	14
a	b	a	b	a	b	a	b	a	a	b	a	b	b
a	b	a	b	b									
→	a	b	a	b	b								
	→	a	b	a	b	b							

Algoritmo Simples (Ingênuo): Ilustração

Ex: **T** = "ababababababb" e **P** = "ababb"

1	2	3	4	5	6	7	8	9	10	11	12	13	14
a	b	a	b	a	b	a	b	a	a	b	a	b	b
a	b	a	b	b									
→	a	b	a	b	b								
	→	a	b	a	b	b							
		→	a	b	a	b	b						

Algoritmo Simples (Ingênuo): Ilustração

Ex: **T** = "ababababababb" e **P** = "ababb"

1	2	3	4	5	6	7	8	9	10	11	12	13	14
a	b	a	b	a	b	a	b	a	a	b	a	b	b
a	b	a	b	b									
→	a	b	a	b	b								
	→	a	b	a	b	b							
		→	a	b	a	b	b						
			→	a	b	a	b	b					

Algoritmo Simples (Ingênuo): Ilustração

Ex: **T** = "ababababababb" e **P** = "ababb"

1	2	3	4	5	6	7	8	9	10	11	12	13	14
a	b	a	b	a	b	a	b	a	a	b	a	b	b
a	b	a	b	b									
→	a	b	a	b	b								
	→	a	b	a	b	b							
		→	a	b	a	b	b						
			→	a	b	a	b	b					
				→	a	b	a	b	b				

Algoritmo Simples (Ingênuo): Ilustração

Ex: **T** = "ababababababb" e **P** = "ababb"

1	2	3	4	5	6	7	8	9	10	11	12	13	14
a	b	a	b	a	b	a	b	a	a	b	a	b	b
a	b	a	b	b									
→	a	b	a	b	b								
	→	a	b	a	b	b							
		→	a	b	a	b	b						
			→	a	b	a	b	b					
				→	a	b	a	b	b				
					→	a	b	a	b	b			

Algoritmo Simples (Ingênuo): Ilustração

Ex: **T** = "ababababababb" e **P** = "ababb"

1	2	3	4	5	6	7	8	9	10	11	12	13	14
a	b	a	b	a	b	a	b	a	a	b	a	b	b
a	b	a	b	b									
→	a	b	a	b	b								
	→	a	b	a	b	b							
		→	a	b	a	b	b						
			→	a	b	a	b	b					
				→	a	b	a	b	b				
					→	a	b	a	b	b			
						→	a	b	a	b	b		
							→	a	b	a	b	b	

Algoritmo Simples (Ingênuo): Ilustração

Ex: **T** = "ababababababb" e **P** = "ababb"

1	2	3	4	5	6	7	8	9	10	11	12	13	14
a	b	a	b	a	b	a	b	a	a	b	a	b	b
a	b	a	b	b									
→	a	b	a	b	b								
	→	a	b	a	b	b							
		→	a	b	a	b	b						
			→	a	b	a	b	b					
				→	a	b	a	b	b				
					→	a	b	a	b	b			
						→	a	b	a	b	b		
							→	a	b	a	b	b	

Algoritmo Simples (Ingênuo): Ilustração

Ex: **T** = "ababababababb" e **P** = "ababb"

1	2	3	4	5	6	7	8	9	10	11	12	13	14
a	b	a	b	a	b	a	b	a	a	b	a	b	b
a	b	a	b	b									
→	a	b	a	b	b								
	→	a	b	a	b	b							
		→	a	b	a	b	b						
			→	a	b	a	b	b					
				→	a	b	a	b	b				
					→	a	b	a	b	b			
						→	a	b	a	b	b		
							→	a	b	a	b	b	
								→	a	b	a	b	b

Algoritmo Simples

Busca de $P[1 \dots m]$ em $T[1 \dots n]$, $m \leq n$.

```
1 int simpleStrMatcher(char T[], char P[]) {
2     int i, j, m, n;
3     ~|
4     ~|n = strlen(T);
5     ~|m = strlen(P);
6     ~|for (i = 0; i <= n-m; i++) {
7     ~|~|for (j = 1; j <= m; j++) {
8     ~|~|~|if (P[j] != T[i+j]) break;
9     ~|~|}
10    ~|~|if (j == m+1) return (i+1);
11    ~|}
12    ~|return (-1);
13 }
```

Complexidade $O(nm)$ – exemplo?

É possível melhorar pré-processando o padrão p

Algoritmo baseado em Autômato

Comparações Desnecessárias

Ex: $T = \text{"ababababaababb"}$ e $P = \text{"ababb"}$

À medida que se conhece o texto, as seguintes comparações em laranja abaixo são desnecessárias:

já se sabe o resultado da comparação *a priori*!

1	2	3	4	5	6	7	8	9	10	11	12	13	14
a	b	a	b	a	b	a	b	a	a	b	a	b	b

Comparações Desnecessárias

Ex: $T = \text{"ababababababb"}$ e $P = \text{"ababb"}$

À medida que se conhece o texto, as seguintes comparações em laranja abaixo são desnecessárias:

já se sabe o resultado da comparação *a priori*!

1	2	3	4	5	6	7	8	9	10	11	12	13	14
a	b	a	b	a	b	a	b	a	a	b	a	b	b
a	b	a	b	b									

Comparações Desnecessárias

Ex: $T = \text{"ababababababb"}$ e $P = \text{"ababb"}$

À medida que se conhece o texto, as seguintes comparações em laranja abaixo são desnecessárias:

já se sabe o resultado da comparação *a priori*!

1	2	3	4	5	6	7	8	9	10	11	12	13	14
a	b	a	b	a	b	a	b	a	a	b	a	b	b
a	b	a	b	b									
→	a												

Comparações Desnecessárias

Ex: $T = \text{"ababababaababb"}$ e $P = \text{"ababb"}$

À medida que se conhece o texto, as seguintes comparações em laranja abaixo são desnecessárias:

já se sabe o resultado da comparação *a priori*!

1	2	3	4	5	6	7	8	9	10	11	12	13	14
a	b	a	b	a	b	a	b	a	a	b	a	b	b
a	b	a	b	b									
→	a												
	→	a	b	a	b	b							

Comparações Desnecessárias

Ex: $T = \text{"ababababaababb"}$ e $P = \text{"ababb"}$

À medida que se conhece o texto, as seguintes comparações em laranja abaixo são desnecessárias:

já se sabe o resultado da comparação *a priori*!

1	2	3	4	5	6	7	8	9	10	11	12	13	14
a	b	a	b	a	b	a	b	a	a	b	a	b	b
a	b	a	b	b									
→	a												
	→	a	b	a	b	b							
		→	a										

Comparações Desnecessárias

Ex: T = “ababababaababb” e P = “ababb”

À medida que se conhece o texto, as seguintes comparações em laranja abaixo são desnecessárias:

já se sabe o resultado da comparação *a priori*!

1	2	3	4	5	6	7	8	9	10	11	12	13	14
a	b	a	b	a	b	a	b	a	a	b	a	b	b
a	b	a	b	b									
→	a												
	→	a	b	a	b	b							
		→	a										
			→	a	b	a	b	b					

Comparações Desnecessárias

Ex: T = “ababababaababb” e P = “ababb”

À medida que se conhece o texto, as seguintes comparações em laranja abaixo são desnecessárias:

já se sabe o resultado da comparação *a priori*!

1	2	3	4	5	6	7	8	9	10	11	12	13	14
a	b	a	b	a	b	a	b	a	a	b	a	b	b
a	b	a	b	b									
→	a												
	→	a	b	a	b	b							
		→	a										
			→	a	b	a	b	b					
				→	a								

Comparações Desnecessárias

Ex: T = “ababababaababb” e P = “ababb”

À medida que se conhece o texto, as seguintes comparações em laranja abaixo são desnecessárias:

já se sabe o resultado da comparação *a priori*!

1	2	3	4	5	6	7	8	9	10	11	12	13	14
a	b	a	b	a	b	a	b	a	a	b	a	b	b
a	b	a	b	b									
→	a												
	→	a	b	a	b	b							
		→	a										
			→	a	b	a	b	b					
				→	a								
					→	a	b	a	b				

Comparações Desnecessárias

Ex: T = “ababababaababb” e P = “ababb”

À medida que se conhece o texto, as seguintes comparações em laranja abaixo são desnecessárias:

já se sabe o resultado da comparação *a priori*!

1	2	3	4	5	6	7	8	9	10	11	12	13	14
a	b	a	b	a	b	a	b	a	a	b	a	b	b
a	b	a	b	b									
→	a												
	→	a	b	a	b	b							
		→	a										
			→	a	b	a	b	b					
				→	a								
					→	a	b	a	b				
						→	a						

Comparações Desnecessárias

Ex: T = “ababababaababb” e P = “ababb”

À medida que se conhece o texto, as seguintes comparações em laranja abaixo são desnecessárias:

já se sabe o resultado da comparação *a priori*!

1	2	3	4	5	6	7	8	9	10	11	12	13	14
a	b	a	b	a	b	a	b	a	a	b	a	b	b
a	b	a	b	b									
→	a												
	→	a	b	a	b	b							
		→	a										
			→	a	b	a	b	b					
				→	a								
					→	a	b	a	b				
						→	a						
							→	a	b				

Comparações Desnecessárias

Ex: T = “ababababaababb” e P = “ababb”

À medida que se conhece o texto, as seguintes comparações em laranja abaixo são desnecessárias:

já se sabe o resultado da comparação *a priori*!

1	2	3	4	5	6	7	8	9	10	11	12	13	14
a	b	a	b	a	b	a	b	a	a	b	a	b	b
a	b	a	b	b									
→	a												
	→	a	b	a	b	b							
		→	a										
			→	a	b	a	b	b					
				→	a								
					→	a	b	a	b				
						→	a						
							→	a	b				
								→	a	b	a	b	b

O que fazer para evitar as comparações desnecessárias?

Princípio geral do Algoritmo baseado em Autômatos:

Uma vez que se conhece o texto até uma posição i , com qual caractere do padrão deve-se comparar **o próximo caractere do texto**?

1	2	3	4	5	6	7	8	9	10	11	12	13	14
a	b	a	b	a	b	a	b	a	a	b	a	b	b

O que fazer para evitar as comparações desnecessárias?

Princípio geral do Algoritmo baseado em Autômatos:

Uma vez que se conhece o texto até uma posição i , com qual caractere do padrão deve-se comparar **o próximo caractere do texto**?

1	2	3	4	5	6	7	8	9	10	11	12	13	14
a	b	a	b	a	b	a	b	a	a	b	a	b	b
a	b	a	b	b									

O que fazer para evitar as comparações desnecessárias?

Princípio geral do Algoritmo baseado em Autômatos:

Uma vez que se conhece o texto até uma posição i , com qual caractere do padrão deve-se comparar **o próximo caractere do texto**?

1	2	3	4	5	6	7	8	9	10	11	12	13	14
a	b	a	b	a	b	a	b	a	a	b	a	b	b
a	b	a	b	b									
	a	b	a	b	b	← Deslocamento de uma posição? Não!							

O que fazer para evitar as comparações desnecessárias?

Princípio geral do Algoritmo baseado em Autômatos:

Uma vez que se conhece o texto até uma posição i , com qual caractere do padrão deve-se comparar **o próximo caractere do texto**?

1	2	3	4	5	6	7	8	9	10	11	12	13	14
a	b	a	b	a	b	a	b	a	a	b	a	b	b
a	b	a	b	b									
	a	b	a	b	b								
		a	b	a	b	b							

← Deslocamento de uma posição? Não!

← De duas? **Sim!**

Pré-processamento do Padrão

Padrão:

$P = a\ b\ a\ b\ a\ c\ a$

Conjunto de caracteres que podem ocorrer no texto: $\Sigma = \{a, b, c\}$

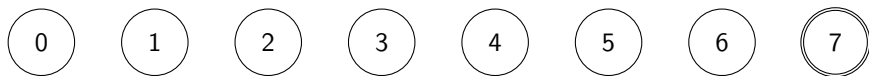
Pré-processamento do Padrão

Padrão:

$P = a b a b a c a$

Conjunto de caracteres que podem ocorrer no texto: $\Sigma = \{a, b, c\}$

Inicialmente, criam-se estados de 0 até o tamanho do prefixo (7, para o exemplo):



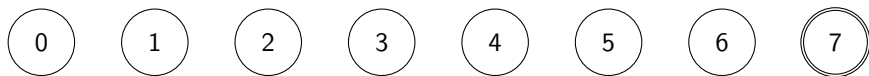
Pré-processamento do Padrão

Padrão:

$P = a b a b a c a$

Conjunto de caracteres que podem ocorrer no texto: $\Sigma = \{a, b, c\}$

Inicialmente, criam-se estados de 0 até o tamanho do prefixo (7, para o exemplo):



Cada estado do autômato indica **o tamanho do prefixo do padrão que casa com o sufixo conhecido do texto**

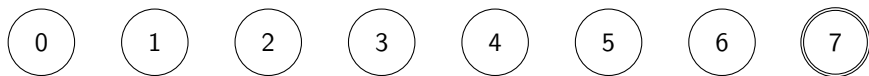
Pré-processamento do Padrão

Padrão:

$P = a\ b\ a\ b\ a\ c\ a$

Conjunto de caracteres que podem ocorrer no texto: $\Sigma = \{a, b, c\}$

Inicialmente, criam-se estados de 0 até o tamanho do prefixo (7, para o exemplo):



Cada estado do autômato indica **o tamanho do prefixo do padrão que casa com o sufixo conhecido do texto**

Para cada estado, verifica-se para qual estado se vai, considerando-se cada possível caractere do texto.

Pré-processamento do Padrão

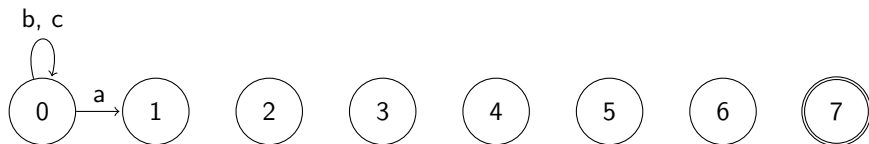
Padrão:

$P = a b a b a c a$

Conjunto de caracteres que podem ocorrer no texto: $\Sigma = \{a, b, c\}$

Para o caso do estado **0**:

- ▶ se o caractere do texto for **a**: vai-se para o estado 1 (expandiu-se o prefixo)
- ▶ se o caractere do texto for **b** ou **c**: permanece-se no estado 0 (não se expandiu o prefixo)



Pré-processamento do Padrão

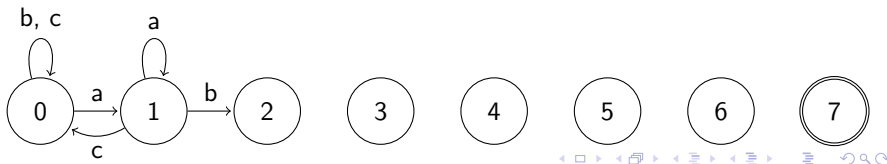
Padrão:

$P = a b a b a c a$

Conjunto de caracteres que podem ocorrer no texto: $\Sigma = \{a, b, c\}$

Para o caso do estado **1**:

- ▶ se o caractere do texto for **a**: permanece-se no estado 1 (não se expandiu o prefixo)
- ▶ se o caractere do texto for **b**: vai para estado 2 (expandiu-se o prefixo)
- ▶ se o caractere do texto for **c**: volta-se para o estado 0 (não há prefixo igual ao sufixo da parte conhecida do texto)



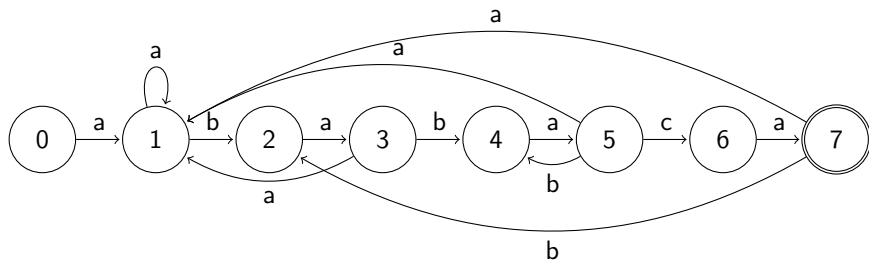
Pré-processamento do Padrão

Padrão:

$P = a\ b\ a\ b\ a\ c\ a$

Conjunto de caracteres que podem ocorrer no texto: $\Sigma = \{a, b, c\}$

Repetindo-se o processo para todos os estados (para clareza, transições para o estado 0 não são mostradas):



Pré-processamento do Padrão: Tabela δ

Representação do autômato na tabela δ :

		Caracteres		
		a	b	c
Estados:	0	1	0	0
	1	1	2	0
	2	3	0	0
	3	1	4	0
	4	5	0	0
	5	1	4	6
	6	7	0	0
	7	1	2	0

Busca do Padrão no Texto

Tabela δ :

Estados:	Caracteres		
	a	b	c
0	1	0	0
1	1	2	0
2	3	0	0
3	1	4	0
4	5	0	0
5	1	4	6
6	7	0	0
7	1	2	0

1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
a	b	a	a	a	b	a	b	a	b	a	c	a	b	b	a

s: 0

Busca do Padrão no Texto

Tabela δ :

		Caracteres		
		a	b	c
Estados:	0	1	0	0
	1	1	2	0
	2	3	0	0
	3	1	4	0
	4	5	0	0
	5	1	4	6
	6	7	0	0
	7	1	2	0

1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
a	b	a	a	a	b	a	b	a	b	a	c	a	b	b	a

s: 0 1

Busca do Padrão no Texto

Tabela δ :

		Caracteres		
		a	b	c
Estados:	0	1	0	0
	1	1	2	0
	2	3	0	0
	3	1	4	0
	4	5	0	0
	5	1	4	6
	6	7	0	0
	7	1	2	0

1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
a	b	a	a	a	b	a	b	a	b	a	c	a	b	b	a

s: 0 1 2

Busca do Padrão no Texto

Tabela δ :

		Caracteres		
		a	b	c
Estados:	0	1	0	0
	1	1	2	0
	2	3	0	0
	3	1	4	0
	4	5	0	0
	5	1	4	6
	6	7	0	0
	7	1	2	0

1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
a	b	a	a	a	b	a	b	a	b	a	c	a	b	b	a

s: 0 1 2 3

Busca do Padrão no Texto

Tabela δ :

		Caracteres		
		a	b	c
Estados:	0	1	0	0
	1	1	2	0
	2	3	0	0
	3	1	4	0
	4	5	0	0
	5	1	4	6
	6	7	0	0
	7	1	2	0

1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
a	b	a	a	a	b	a	b	a	b	a	c	a	b	b	a

s: 0 1 2 3 1

Busca do Padrão no Texto

Tabela δ :

		Caracteres		
		a	b	c
Estados:	0	1	0	0
	1	1	2	0
	2	3	0	0
	3	1	4	0
	4	5	0	0
	5	1	4	6
	6	7	0	0
	7	1	2	0

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
	a	b	a	a	a	b	a	b	a	b	a	c	a	b	b	a
s: 0	1	2	3	1	1	2	3	4	5	4	5	6	7	2	0	1

Revedo as Comparações Desnecessárias

Ex: T = “abaaabababaca” (até o padrão) e P = “ababaca”

Comparações desnecessárias em relação ao algoritmo simples (em laranja):

1	2	3	4	5	6	7	8	9	10	11	12	13
a	b	a	a	a	b	a	b	a	b	a	c	a
a	b	a	b									
	a											
		a	b									
			a	b								
				a	b	a	b	a	c			
					a							
						a	b	a	b	a	c	a

Algoritmo para Calcular a Função de Transição

$P[1..m]$: padrão de tamanho m

P_k : prefixo de P de tamanho k , $1 \leq k \leq m$

$P_k \cdot a$: concatenação de P_k com caractere a

Σ : alfabeto de caracteres que podem ocorrer no texto

Algorithm 1: Algoritmo para calcular a função de transição δ

entrada : Padrão $P[1..m]$ e alfabeto Σ

saida : Função de transição δ

```
1  $m \leftarrow P.length;$ 
2 for  $q = 0$  to  $m$  do
3   for each character  $a \in \Sigma$  do
4      $k \leftarrow \min(m + 1, q + 2);$ 
5     repeat
6        $k \leftarrow k - 1;$ 
7     until  $P_k$  is a suffix of  $P_q \cdot a;$ 
8      $\delta(q, a) \leftarrow k;$ 
9 return  $\delta;$ 
```

Algoritmo para Calcular a Função de Transição

$P[1..m]$: padrão de tamanho m

P_k : prefixo de P de tamanho k , $1 \leq k \leq m$

$P_k \cdot a$: concatenação de P_k com caractere a

Σ : alfabeto de caracteres que podem ocorrer no texto

Algorithm 2: Algoritmo para calcular a função de transição δ

entrada : Padrão $P[1..m]$ e alfabeto Σ

saida : Função de transição δ

```
1  $m \leftarrow P.length;$ 
2 for  $q = 0$  to  $m$  do
3   for each character  $a \in \Sigma$  do
4      $k \leftarrow \min(m + 1, q + 2);$ 
5     repeat
6        $k \leftarrow k - 1;$ 
7     until  $P_k$  is a suffix of  $P_q \cdot a;$ 
8      $\delta(q, a) \leftarrow k;$ 
9 return  $\delta;$ 
```

Complexidade: $O(m^3|\Sigma|)$. Mas δ pode ser calculada em $O(m|\Sigma|)$

Algoritmo para Encontrar o Padrão no Texto

Algorithm 3: Algoritmo para encontrar o padrão no texto

entrada : Texto $T[1..n]$, Função de Transição δ , tamanho do padrão m

```
1  $n \leftarrow T.length;$ 
2  $q \leftarrow 0;$ 
3 for  $i = 1$  to  $n$  do
4    $q \leftarrow \delta(q, T[i]);$ 
5   if  $q = m$  then
6      $\text{print "Padrão ocorre com deslocamento"} i - m;$ 
```

Algoritmo para Encontrar o Padrão no Texto

Algorithm 4: Algoritmo para encontrar o padrão no texto

entrada : Texto $T[1..n]$, Função de Transição δ , tamanho do padrão m

```
1  $n \leftarrow T.length;$ 
2  $q \leftarrow 0;$ 
3 for  $i = 1$  to  $n$  do
4    $q \leftarrow \delta(q, T[i]);$ 
5   if  $q = m$  then
6     print "Padrão ocorre com deslocamento"  $i - m;$ 
```

Complexidade: $\Theta(n)$

Exercício

Qual seria a tabela δ e quais seriam as transições de estado para se procurar o padrão P no texto T abaixo:

$T = \text{abababcbaababababcbab}$

$P = \text{ababcbab}$

Considere que o alfabeto $\Sigma = \{a, b, c\}$.

Exercício: Resposta

	δ		
	a	b	c
0	1	0	0
1	1	2	0
2	3	0	0
3	1	4	0
4	3	0	5
5	1	6	0
6	7	0	0
7	1	8	0
7	3	0	0

	1	2	3	4	5	6	7	8	9	10	11	12
	a	b	a	b	a	b	c	b	a	a	b	a
s: 0	1	2	3	4	3	4	5	6	7	1	2	3

	13	14	15	16	17	18	19	20	21
	b	a	b	a	b	c	b	a	b
s:	4	3	4	3	4	5	6	7	8

Como diminuir a complexidade do cálculo de δ ?

- ▶ A tabela δ inclui todos os elementos do alfabeto Σ

Como diminuir a complexidade do cálculo de δ ?

- ▶ A tabela δ inclui todos os elementos do alfabeto Σ
- ▶ Para cada estado, consideram-se todos os possíveis elementos do alfabeto $\rightarrow m \cdot |\Sigma|$

Como diminuir a complexidade do cálculo de δ ?

- ▶ A tabela δ inclui todos os elementos do alfabeto Σ
- ▶ Para cada estado, consideram-se todos os possíveis elementos do alfabeto $\rightarrow m \cdot |\Sigma|$
- ▶ Idéia para diminuir a complexidade: não considerar o caractere atual do texto ao se pré-processar o padrão $\rightarrow$ **apenas m**

Como diminuir a complexidade do cálculo de δ ?

- ▶ A tabela δ inclui todos os elementos do alfabeto Σ
- ▶ Para cada estado, consideram-se todos os possíveis elementos do alfabeto $\rightarrow m \cdot |\Sigma|$
- ▶ Idéia para diminuir a complexidade: não considerar o caractere atual do texto ao se pré-processar o padrão $\rightarrow$ **apenas m**
- ▶ Essa é a idéia do **KMP!**

Algoritmo KMP (Knuth-Morris-Pratt)

KMP: Ilustração

Ex: $T = \text{"ababababababb"}$ e $P = \text{"ababb"}$

1	2	3	4	5	6	7	8	9	10	11	12	13	14
a	b	a	b	a	b	a	b	a	a	b	a	b	b
a	b	a	b	b									
	→	a	b	a	b	b							
		→	a	b	a	b	b						
			→	a	b	a	b	b					
				→	a	b	a	b	b	b			
					→	a	b	a	b	a	b	b	
						→	a	b	a	b	a	b	b

KMP: Pré-processamento do Padrão

Objetivo

Para cada posição j de P , encontrar o tamanho do maior sufixo que é igual a um prefixo de $P[1 \dots j]$ (de tamanho menor que j)

Ex: $P = \text{"ababbababa"}$

j	1	2	3	4	5	6	7	8	9	10
P	a	b	a	b	b	a	b	a	b	a
π										

KMP: Pré-processamento do Padrão

Objetivo

Para cada posição j de P , encontrar o tamanho do maior sufixo que é igual a um prefixo de $P[1 \dots j]$ (de tamanho menor que j)

Ex: $P = \text{"ababbababa"}$

j	1	2	3	4	5	6	7	8	9	10
P	a	b	a	b	b	a	b	a	b	a
π	0									

KMP: Pré-processamento do Padrão

Objetivo

Para cada posição j de P , encontrar o tamanho do maior sufixo que é igual a um prefixo de $P[1 \dots j]$ (de tamanho menor que j)

Ex: $P = \text{"ababbababa"}$

j	1	2	3	4	5	6	7	8	9	10
P	a	b	a	b	b	a	b	a	b	a
π	0	0								

KMP: Pré-processamento do Padrão

Objetivo

Para cada posição j de P , encontrar o tamanho do maior sufixo que é igual a um prefixo de $P[1 \dots j]$ (de tamanho menor que j)

Ex: $P = \text{"ababbababa"}$

j	1	2	3	4	5	6	7	8	9	10
P	a	b	a	b	b	a	b	a	b	a
π	0	0	1							

KMP: Pré-processamento do Padrão

Objetivo

Para cada posição j de P , encontrar o tamanho do maior sufixo que é igual a um prefixo de $P[1 \dots j]$ (de tamanho menor que j)

Ex: $P = \text{"ababbababa"}$

j	1	2	3	4	5	6	7	8	9	10
P	a	b	a	b	b	a	b	a	b	a
π	0	0	1	2						

KMP: Pré-processamento do Padrão

Objetivo

Para cada posição j de P , encontrar o tamanho do maior sufixo que é igual a um prefixo de $P[1 \dots j]$ (de tamanho menor que j)

Ex: $P = \text{"ababbababa"}$

j	1	2	3	4	5	6	7	8	9	10
P	a	b	a	b	b	a	b	a	b	a
π	0	0	1	2	0					

KMP: Pré-processamento do Padrão

Objetivo

Para cada posição j de P , encontrar o tamanho do maior sufixo que é igual a um prefixo de $P[1 \dots j]$ (de tamanho menor que j)

Ex: $P = \text{"ababbababa"}$

j	1	2	3	4	5	6	7	8	9	10
P	a	b	a	b	b	a	b	a	b	a
π	0	0	1	2	0	1				

KMP: Pré-processamento do Padrão

Objetivo

Para cada posição j de P , encontrar o tamanho do maior sufixo que é igual a um prefixo de $P[1 \dots j]$ (de tamanho menor que j)

Ex: $P = \text{"ababbababa"}$

j	1	2	3	4	5	6	7	8	9	10
P	a	b	a	b	b	a	b	a	b	a
π	0	0	1	2	0	1	2			

KMP: Pré-processamento do Padrão

Objetivo

Para cada posição j de P , encontrar o tamanho do maior sufixo que é igual a um prefixo de $P[1 \dots j]$ (de tamanho menor que j)

Ex: $P = \text{"ababbababa"}$

j	1	2	3	4	5	6	7	8	9	10
P	a	b	a	b	b	a	b	a	b	a
π	0	0	1	2	0	1	2	3		

KMP: Pré-processamento do Padrão

Objetivo

Para cada posição j de P , encontrar o tamanho do maior sufixo que é igual a um prefixo de $P[1 \dots j]$ (de tamanho menor que j)

Ex: $P = \text{"ababbababa"}$

j	1	2	3	4	5	6	7	8	9	10
P	a	b	a	b	b	a	b	a	b	a
π	0	0	1	2	0	1	2	3	4	

KMP: Pré-processamento do Padrão

Objetivo

Para cada posição j de P , encontrar o tamanho do maior sufixo que é igual a um prefixo de $P[1 \dots j]$ (de tamanho menor que j)

Ex: $P = \text{"ababbababa"}$

j	1	2	3	4	5	6	7	8	9	10
P	a	b	a	b	b	a	b	a	b	a
π	0	0	1	2	0	1	2	3	4	3

KMP: Ilustração

Ex: $T = \text{"abaaabababaca"}$ e $P = \text{"ababaca"}$

j	1	2	3	4	5	6	7
P	a	b	a	b	a	c	a
π	0	0	1	2	3	0	1

1	2	3	4	5	6	7	8	9	10	11	12	13
a	b	a	a	a	b	a	b	a	b	a	c	a
a	b	a	b									
		a	b									
			a	b								
				a	b	a	b	a	c			
						a	b	a	b	a	c	a

KMP: Ilustração

Ex: $T = \text{"abaaabababaca"}$ e $P = \text{"ababaca"}$

j	1	2	3	4	5	6	7
P	a	b	a	b	a	c	a
π	0	0	1	2	3	0	1

1	2	3	4	5	6	7	8	9	10	11	12	13
a	b	a	a	a	b	a	b	a	b	a	c	a
a	b	a	b									
		a	b									
			a	b								
				a	b	a	b	a	c			
						a	b	a	b	a	c	a

Qual é a lógica de definição da próxima comparação?

KMP: Pré-processamento do Padrão

Idéia geral:

O valor de $\pi[1] = 0$. Para se determinar o valor de $\pi[i]$, $i > 1$:

KMP: Pré-processamento do Padrão

Idéia geral:

O valor de $\pi[1] = 0$. Para se determinar o valor de $\pi[i]$, $i > 1$:

- ▶ verifica-se se é possível "estender" o maior prefixo igual a sufixo encontrado para a posição $i - 1$, ou seja, se $P[\pi[i - 1] + 1] = P[i]$

KMP: Pré-processamento do Padrão

Idéia geral:

O valor de $\pi[1] = 0$. Para se determinar o valor de $\pi[i]$, $i > 1$:

- ▶ verifica-se se é possível "estender" o maior prefixo igual a sufixo encontrado para a posição $i - 1$, ou seja, se $P[\pi[i - 1] + 1] = P[i]$
 - ▶ se sim, $\pi[i] \leftarrow \pi[i - 1] + 1$

KMP: Pré-processamento do Padrão

Idéia geral:

O valor de $\pi[1] = 0$. Para se determinar o valor de $\pi[i]$, $i > 1$:

- ▶ verifica-se se é possível "estender" o maior prefixo igual a sufixo encontrado para a posição $i - 1$, ou seja, se $P[\pi[i - 1] + 1] = P[i]$
 - ▶ se sim, $\pi[i] \leftarrow \pi[i - 1] + 1$
 - ▶ se não, verifica-se se é possível "estender" o próximo maior prefixo igual a sufixo, utilizando o valor de $\pi[\pi[i - 1]]$

KMP: Pré-processamento do Padrão

Idéia geral:

O valor de $\pi[1] = 0$. Para se determinar o valor de $\pi[i]$, $i > 1$:

- ▶ verifica-se se é possível "estender" o maior prefixo igual a sufixo encontrado para a posição $i - 1$, ou seja, se $P[\pi[i - 1] + 1] = P[i]$
 - ▶ se sim, $\pi[i] \leftarrow \pi[i - 1] + 1$
 - ▶ se não, verifica-se se é possível "estender" o próximo maior prefixo igual a sufixo, utilizando o valor de $\pi[\pi[i - 1]]$
 - ▶ repete-se o processo até ser possível "estender" um prefixo igual a sufixo ou se chegar ao fim da tabela

<i>j</i>	1	2	3	4	5	6	7	8	9	10	11
<i>P</i>	a	b	a	b	a	a	b	a	b	c	a
π											

KMP: Pré-processamento do Padrão

Idéia geral:

O valor de $\pi[1] = 0$. Para se determinar o valor de $\pi[i]$, $i > 1$:

- ▶ verifica-se se é possível "estender" o maior prefixo igual a sufixo encontrado para a posição $i - 1$, ou seja, se $P[\pi[i - 1] + 1] = P[i]$
 - ▶ se sim, $\pi[i] \leftarrow \pi[i - 1] + 1$
 - ▶ se não, verifica-se se é possível "estender" o próximo maior prefixo igual a sufixo, utilizando o valor de $\pi[\pi[i - 1]]$
 - ▶ repete-se o processo até ser possível "estender" um prefixo igual a sufixo ou se chegar ao fim da tabela

<i>j</i>	1	2	3	4	5	6	7	8	9	10	11
<i>P</i>	a	b	a	b	a	a	b	a	b	c	a
π	0										

KMP: Pré-processamento do Padrão

Idéia geral:

O valor de $\pi[1] = 0$. Para se determinar o valor de $\pi[i]$, $i > 1$:

- ▶ verifica-se se é possível "estender" o maior prefixo igual a sufixo encontrado para a posição $i - 1$, ou seja, se $P[\pi[i - 1] + 1] = P[i]$
 - ▶ se sim, $\pi[i] \leftarrow \pi[i - 1] + 1$
 - ▶ se não, verifica-se se é possível "estender" o próximo maior prefixo igual a sufixo, utilizando o valor de $\pi[\pi[i - 1]]$
 - ▶ repete-se o processo até ser possível "estender" um prefixo igual a sufixo ou se chegar ao fim da tabela

<i>j</i>	1	2	3	4	5	6	7	8	9	10	11
<i>P</i>	a	b	a	b	a	a	b	a	b	c	a
π	0	0									

KMP: Pré-processamento do Padrão

Idéia geral:

O valor de $\pi[1] = 0$. Para se determinar o valor de $\pi[i]$, $i > 1$:

- ▶ verifica-se se é possível "estender" o maior prefixo igual a sufixo encontrado para a posição $i - 1$, ou seja, se $P[\pi[i - 1] + 1] = P[i]$
 - ▶ se sim, $\pi[i] \leftarrow \pi[i - 1] + 1$
 - ▶ se não, verifica-se se é possível "estender" o próximo maior prefixo igual a sufixo, utilizando o valor de $\pi[\pi[i - 1]]$
 - ▶ repete-se o processo até ser possível "estender" um prefixo igual a sufixo ou se chegar ao fim da tabela

<i>j</i>	1	2	3	4	5	6	7	8	9	10	11
<i>P</i>	a	b	a	b	a	a	b	a	b	c	a
π	0	0	1								

KMP: Pré-processamento do Padrão

Idéia geral:

O valor de $\pi[1] = 0$. Para se determinar o valor de $\pi[i]$, $i > 1$:

- ▶ verifica-se se é possível "estender" o maior prefixo igual a sufixo encontrado para a posição $i - 1$, ou seja, se $P[\pi[i - 1] + 1] = P[i]$
 - ▶ se sim, $\pi[i] \leftarrow \pi[i - 1] + 1$
 - ▶ se não, verifica-se se é possível "estender" o próximo maior prefixo igual a sufixo, utilizando o valor de $\pi[\pi[i - 1]]$
 - ▶ repete-se o processo até ser possível "estender" um prefixo igual a sufixo ou se chegar ao fim da tabela

<i>j</i>	1	2	3	4	5	6	7	8	9	10	11
<i>P</i>	a	b	a	b	a	a	b	a	b	c	a
π	0	0	1	2							

KMP: Pré-processamento do Padrão

Idéia geral:

O valor de $\pi[1] = 0$. Para se determinar o valor de $\pi[i]$, $i > 1$:

- ▶ verifica-se se é possível "estender" o maior prefixo igual a sufixo encontrado para a posição $i - 1$, ou seja, se $P[\pi[i - 1] + 1] = P[i]$
 - ▶ se sim, $\pi[i] \leftarrow \pi[i - 1] + 1$
 - ▶ se não, verifica-se se é possível "estender" o próximo maior prefixo igual a sufixo, utilizando o valor de $\pi[\pi[i - 1]]$
 - ▶ repete-se o processo até ser possível "estender" um prefixo igual a sufixo ou se chegar ao fim da tabela

j	1	2	3	4	5	6	7	8	9	10	11
P	a	b	a	b	a	a	b	a	b	c	a
π	0	0	1	2	3						

KMP: Pré-processamento do Padrão

Idéia geral:

O valor de $\pi[1] = 0$. Para se determinar o valor de $\pi[i]$, $i > 1$:

- ▶ verifica-se se é possível "estender" o maior prefixo igual a sufixo encontrado para a posição $i - 1$, ou seja, se $P[\pi[i - 1] + 1] = P[i]$
 - ▶ se sim, $\pi[i] \leftarrow \pi[i - 1] + 1$
 - ▶ se não, verifica-se se é possível "estender" o próximo maior prefixo igual a sufixo, utilizando o valor de $\pi[\pi[i - 1]]$
 - ▶ repete-se o processo até ser possível "estender" um prefixo igual a sufixo ou se chegar ao fim da tabela

j	1	2	3	4	5	6	7	8	9	10	11
P	a	b	a	b	a	a	b	a	b	c	a
π	0	0	1	2	3	1					

KMP: Pré-processamento do Padrão

Idéia geral:

O valor de $\pi[1] = 0$. Para se determinar o valor de $\pi[i]$, $i > 1$:

- ▶ verifica-se se é possível "estender" o maior prefixo igual a sufixo encontrado para a posição $i - 1$, ou seja, se $P[\pi[i - 1] + 1] = P[i]$
 - ▶ se sim, $\pi[i] \leftarrow \pi[i - 1] + 1$
 - ▶ se não, verifica-se se é possível "estender" o próximo maior prefixo igual a sufixo, utilizando o valor de $\pi[\pi[i - 1]]$
 - ▶ repete-se o processo até ser possível "estender" um prefixo igual a sufixo ou se chegar ao fim da tabela

<i>j</i>	1	2	3	4	5	6	7	8	9	10	11
<i>P</i>	a	b	a	b	a	a	b	a	b	c	a
π	0	0	1	2	3	1	2				

KMP: Pré-processamento do Padrão

Idéia geral:

O valor de $\pi[1] = 0$. Para se determinar o valor de $\pi[i]$, $i > 1$:

- ▶ verifica-se se é possível "estender" o maior prefixo igual a sufixo encontrado para a posição $i - 1$, ou seja, se $P[\pi[i - 1] + 1] = P[i]$
 - ▶ se sim, $\pi[i] \leftarrow \pi[i - 1] + 1$
 - ▶ se não, verifica-se se é possível "estender" o próximo maior prefixo igual a sufixo, utilizando o valor de $\pi[\pi[i - 1]]$
 - ▶ repete-se o processo até ser possível "estender" um prefixo igual a sufixo ou se chegar ao fim da tabela

j	1	2	3	4	5	6	7	8	9	10	11
P	a	b	a	b	a	a	b	a	b	c	a
π	0	0	1	2	3	1	2	3			

KMP: Pré-processamento do Padrão

Idéia geral:

O valor de $\pi[1] = 0$. Para se determinar o valor de $\pi[i]$, $i > 1$:

- ▶ verifica-se se é possível "estender" o maior prefixo igual a sufixo encontrado para a posição $i - 1$, ou seja, se $P[\pi[i - 1] + 1] = P[i]$
 - ▶ se sim, $\pi[i] \leftarrow \pi[i - 1] + 1$
 - ▶ se não, verifica-se se é possível "estender" o próximo maior prefixo igual a sufixo, utilizando o valor de $\pi[\pi[i - 1]]$
 - ▶ repete-se o processo até ser possível "estender" um prefixo igual a sufixo ou se chegar ao fim da tabela

<i>j</i>	1	2	3	4	5	6	7	8	9	10	11
<i>P</i>	a	b	a	b	a	a	b	a	b	c	a
π	0	0	1	2	3	1	2	3	4		

KMP: Pré-processamento do Padrão

Idéia geral:

O valor de $\pi[1] = 0$. Para se determinar o valor de $\pi[i]$, $i > 1$:

- ▶ verifica-se se é possível "estender" o maior prefixo igual a sufixo encontrado para a posição $i - 1$, ou seja, se $P[\pi[i - 1] + 1] = P[i]$
 - ▶ se sim, $\pi[i] \leftarrow \pi[i - 1] + 1$
 - ▶ se não, verifica-se se é possível "estender" o próximo maior prefixo igual a sufixo, utilizando o valor de $\pi[\pi[i - 1]]$
 - ▶ repete-se o processo até ser possível "estender" um prefixo igual a sufixo ou se chegar ao fim da tabela

<i>j</i>	1	2	3	4	5	6	7	8	9	10	11
<i>P</i>	a	b	a	b	a	a	b	a	b	c	a
π	0	0	1	2	3	1	2	3	4	0	

KMP: Pré-processamento do Padrão

Idéia geral:

O valor de $\pi[1] = 0$. Para se determinar o valor de $\pi[i]$, $i > 1$:

- ▶ verifica-se se é possível "estender" o maior prefixo igual a sufixo encontrado para a posição $i - 1$, ou seja, se $P[\pi[i - 1] + 1] = P[i]$
 - ▶ se sim, $\pi[i] \leftarrow \pi[i - 1] + 1$
 - ▶ se não, verifica-se se é possível "estender" o próximo maior prefixo igual a sufixo, utilizando o valor de $\pi[\pi[i - 1]]$
 - ▶ repete-se o processo até ser possível "estender" um prefixo igual a sufixo ou se chegar ao fim da tabela

<i>j</i>	1	2	3	4	5	6	7	8	9	10	11
<i>P</i>	a	b	a	b	a	a	b	a	b	c	a
π	0	0	1	2	3	1	2	3	4	0	1

KMP: Pré-processamento do Padrão

Pergunta de fixação:

Quais comparações o algoritmo KMP realiza para encontrar o valor 0 para a posição $\pi[12]$ e quais os resultados das comparações?

i	1	2	3	4	5	6	7	8	9	10	11	12
p	c_1	c_2	c_3	c_4	c_5	c_6	c_7	c_8	c_9	c_{10}	c_{11}	c_{12}
π	0	0	1	2	0	1	2	3	4	5	6	0

KMP: Pré-processamento do Padrão

Pergunta de fixação:

Quais comparações o algoritmo KMP realiza para encontrar o valor 0 para a posição $\pi[12]$ e quais os resultados das comparações?

i	1	2	3	4	5	6	7	8	9	10	11	12
p	c_1	c_2	c_3	c_4	c_5	c_6	c_7	c_8	c_9	c_{10}	c_{11}	c_{12}
π	0	0	1	2	0	1	2	3	4	5	6	0

$$c_{12} \neq c_7$$

KMP: Pré-processamento do Padrão

Pergunta de fixação:

Quais comparações o algoritmo KMP realiza para encontrar o valor 0 para a posição $\pi[12]$ e quais os resultados das comparações?

i	1	2	3	4	5	6	7	8	9	10	11	12
p	c_1	c_2	c_3	c_4	c_5	c_6	c_7	c_8	c_9	c_{10}	c_{11}	c_{12}
π	0	0	1	2	0	1	2	3	4	5	6	0

$$c_{12} \neq c_2$$

KMP: Pré-processamento do Padrão

Pergunta de fixação:

Quais comparações o algoritmo KMP realiza para encontrar o valor 0 para a posição $\pi[12]$ e quais os resultados das comparações?

i	1	2	3	4	5	6	7	8	9	10	11	12
p	c_1	c_2	c_3	c_4	c_5	c_6	c_7	c_8	c_9	c_{10}	c_{11}	c_{12}
π	0	0	1	2	0	1	2	3	4	5	6	0

$c_{12} \neq c_1$ Portanto $\pi[12] \leftarrow 0$

KMP: Pré-processamento do Padrão

Algorithm 5: COMPUTE_PREFIX_FUNCTION (P)

entrada : Padrão $P[1..m]$

saida : Tabela π

```
1  $m \leftarrow P.length;$ 
2 let  $\pi[1..m]$  be a new array;
3  $\pi[0] \leftarrow 0$ 
4  $q \leftarrow 0;$ 
5 for  $i \leftarrow 2$  to  $m$  do
6   while  $q > 0$  and  $P[q + 1] \neq P[i]$  do
7      $q \leftarrow \pi[q];$ 
8   if  $P[q + 1] = P[i]$  then
9      $q \leftarrow q + 1;$ 
10   $\pi[i] \leftarrow q;$ 
11 return  $\pi;$ 
```

KMP: Pré-processamento do Padrão

Encontre os valores da tabela π para o padrão $P = \text{"ababbababa"}$, seguindo o algoritmo *COMPUTE_PREFIX_FUNCTION*.

Resposta:

j	1	2	3	4	5	6	7	8	9	10
P	a	b	a	b	b	a	b	a	b	a
π										

KMP: Pré-processamento do Padrão

Encontre os valores da tabela π para o padrão $P = \text{"ababbababa"}$, seguindo o algoritmo *COMPUTE_PREFIX_FUNCTION*.

Resposta:

j	1	2	3	4	5	6	7	8	9	10
P	a	b	a	b	b	a	b	a	b	a
π	0									

KMP: Pré-processamento do Padrão

Encontre os valores da tabela π para o padrão $P = \text{"ababbababa"}$, seguindo o algoritmo *COMPUTE_PREFIX_FUNCTION*.

Resposta:

j	1	2	3	4	5	6	7	8	9	10
P	a	b	a	b	b	a	b	a	b	a
π	0	0								

KMP: Pré-processamento do Padrão

Encontre os valores da tabela π para o padrão $P = \text{"ababbababa"}$, seguindo o algoritmo *COMPUTE_PREFIX_FUNCTION*.

Resposta:

j	1	2	3	4	5	6	7	8	9	10
P	a	b	a	b	b	a	b	a	b	a
π	0	0	1							

KMP: Pré-processamento do Padrão

Encontre os valores da tabela π para o padrão $P = \text{"ababbababa"}$, seguindo o algoritmo *COMPUTE_PREFIX_FUNCTION*.

Resposta:

j	1	2	3	4	5	6	7	8	9	10
P	a	b	a	b	b	a	b	a	b	a
π	0	0	1	2						

KMP: Pré-processamento do Padrão

Encontre os valores da tabela π para o padrão $P = \text{"ababbababa"}$, seguindo o algoritmo *COMPUTE_PREFIX_FUNCTION*.

Resposta:

j	1	2	3	4	5	6	7	8	9	10
P	a	b	a	b	b	a	b	a	b	a
π	0	0	1	2	0					

KMP: Pré-processamento do Padrão

Encontre os valores da tabela π para o padrão $P = \text{"ababbababa"}$, seguindo o algoritmo *COMPUTE_PREFIX_FUNCTION*.

Resposta:

j	1	2	3	4	5	6	7	8	9	10
P	a	b	a	b	b	a	b	a	b	a
π	0	0	1	2	0	1				

KMP: Pré-processamento do Padrão

Encontre os valores da tabela π para o padrão $P = \text{"ababbababa"}$, seguindo o algoritmo *COMPUTE_PREFIX_FUNCTION*.

Resposta:

j	1	2	3	4	5	6	7	8	9	10
P	a	b	a	b	b	a	b	a	b	a
π	0	0	1	2	0	1	2			

KMP: Pré-processamento do Padrão

Encontre os valores da tabela π para o padrão $P = \text{"ababbababa"}$, seguindo o algoritmo *COMPUTE_PREFIX_FUNCTION*.

Resposta:

j	1	2	3	4	5	6	7	8	9	10
P	a	b	a	b	b	a	b	a	b	a
π	0	0	1	2	0	1	2	3		

KMP: Pré-processamento do Padrão

Encontre os valores da tabela π para o padrão $P = \text{"ababbababa"}$, seguindo o algoritmo *COMPUTE_PREFIX_FUNCTION*.

Resposta:

j	1	2	3	4	5	6	7	8	9	10
P	a	b	a	b	b	a	b	a	b	a
π	0	0	1	2	0	1	2	3	4	

KMP: Pré-processamento do Padrão

Encontre os valores da tabela π para o padrão $P = \text{"ababbababa"}$, seguindo o algoritmo *COMPUTE_PREFIX_FUNCTION*.

Resposta:

j	1	2	3	4	5	6	7	8	9	10
P	a	b	a	b	b	a	b	a	b	a
π	0	0	1	2	0	1	2	3	4	3

Algorithm 6: *KMP_Matcher*(T, P)

entrada : Texto $T[1..n]$ e Padrão $P[1..m]$

```
1  $n \leftarrow T.length;$ 
2  $m \leftarrow P.length;$ 
3  $\pi \leftarrow COMPUTE\_PREFIX\_FUNCTION(P);$ 
4  $q \leftarrow 0;$ 
5 for  $i = 1$  to  $n$  do
6   while  $q > 0$  and  $P[q + 1] \neq T[i]$  do
7      $q \leftarrow \pi[q];$ 
8   if  $P[q + 1] = T[i]$  then
9      $q \leftarrow q + 1;$ 
10  if  $q = m$  then
11    print "Pattern occurs with shift"  $i - m;$ 
12   $q \leftarrow \pi[q];$ 
```

Exercício

Para o texto e o padrão abaixo:

$T = \text{"aabaaabaabaabaabaababbabaab"}$

$P = \text{"aabaabaa"}$

apresente a tabela π e as comparações realizadas para se encontrar as ocorrências de P em T .

Exercício - Resposta

i	1	2	3	4	5	6	7	8
P	a	a	b	a	a	b	a	a
π	0	1	0	1	2	3	4	5

[illegible]

KMP: Complexidade

- ▶ O pré-processamento do padrão possui complexidade $\Theta(m)$

Argumentos informais:

- ▶ q é sempre positivo e somente se acrescenta seu valor no máximo $m - 1$ vezes (*COMPUTE_PREFIX_FUNCTION*, linha 9)
 - ▶ para cada posição i somente se pode diminuir o valor de q até zero
 - ▶ portanto, não se fazem mais do que $2m$ comparações
-
- ▶ Por argumento análogo, o procedimento de busca é $\Theta(n)$

Algoritmo de Boyer-Moore

Algoritmo Boyer-Moore

Idéia Básica

Ao invés de se comparar os caracteres do texto com os caracteres do padrão a partir do início do padrão ... **compara-se a partir do fim!**

Considere o texto abaixo e o padrão $P = abacabb$, tamanho $m = 7$.

Se compararmos o primeiro caractere do padrão com o primeiro caractere do texto, qual deslocamento podemos fazer?

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20
T :	b	a	b	b	a	b	d	a	b	b	a	a	c	a	b	a	c	a	b	b
	a	b	a	c	a	b	b													

Algoritmo Boyer-Moore

Idéia Básica

Ao invés de se comparar os caracteres do texto com os caracteres do padrão a partir do início do padrão ... **compara-se a partir do fim!**

Considere o texto abaixo e o padrão $P = abacabb$, tamanho $m = 7$.

Se compararmos o primeiro caractere do padrão com o primeiro caractere do texto, qual deslocamento podemos fazer?

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20
T :	b	a	b	b	a	b	d	a	b	b	a	a	c	a	b	a	c	a	b	b
	a	b	a	c	a	b	b													
	→	a	b	a	c	a	b	b												

Apenas avanço de uma posição

Algoritmo Boyer-Moore

Idéia Básica

Ao invés de se comparar os caracteres do texto com os caracteres do padrão a partir do início do padrão ... **compara-se a partir do fim!**

Considere o texto abaixo e o padrão $P = abacabb$, tamanho $m = 7$.

E se compararmos o sétimo elemento do padrão b com o sétimo caractere do texto?

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20
T :	b	a	b	b	a	b	d	a	b	b	a	a	c	a	b	a	c	a	b	b
	a	b	a	c	a	b	b													

Algoritmo Boyer-Moore

Idéia Básica

Ao invés de se comparar os caracteres do texto com os caracteres do padrão a partir do início do padrão ... **compara-se a partir do fim!**

Considere o texto abaixo e o padrão $P = abacabb$, tamanho $m = 7$.

E se compararmos o sétimo elemento do padrão b com o sétimo caractere do texto?

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20
T :	b	a	b	b	a	b	d	a	b	b	a	a	c	a	b	a	c	a	b	b
	a	b	a	c	a	b	b													
							→	a	b	a	c	a	b	b						

Avanço de 7 posições, pois **d** não ocorre no padrão!

Algoritmo Boyer-Moore

Idéia Básica

Ao invés de se comparar os caracteres do texto com os caracteres do padrão a partir do início do padrão ... **compara-se a partir do fim!**

Considere o texto abaixo e o padrão $P = abacabb$, tamanho $m = 7$.

E qual descolamento pode ser feito agora?

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20
$T:$	b	a	b	b	a	b	d	a	b	b	a	a	c	a	b	a	c	a	b	b
	a	b	a	c	a	b	b													
								a	b	a	c	a	b	b						

Algoritmo Boyer-Moore

Idéia Básica

Ao invés de se comparar os caracteres do texto com os caracteres do padrão a partir do início do padrão ... **compara-se a partir do fim!**

Considere o texto abaixo e o padrão $P = abacabb$, tamanho $m = 7$.

E se compararmos o sétimo elemento do padrão b com o sétimo caractere do texto?

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20
T :	b	a	b	b	a	b	d	a	b	b	a	a	c	a	b	a	c	a	b	b
	a	b	a	c	a	b	b													
								a	b	a	c	a	b	b						
								→	a	b	a	c	a	b	b					

Avanço de 2 posições, para se alinhar o **a** do texto (posição 14) com o **a** mais à direita do padrão (posição 5)!

Algoritmo Boyer-Moore

Idéia Básica

Ao invés de se comparar os caracteres do texto com os caracteres do padrão a partir do início do padrão ... **compara-se a partir do fim!**

Considere o texto abaixo e o padrão $P = abacabb$, tamanho $m = 7$.

Seguindo o mesmo princípio:

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20
T :	b	a	b	b	a	b	d	a	b	b	a	a	c	a	b	a	c	a	b	b
	a	b	a	c	a	b	b													
								a	b	a	c	a	b	b						
										a	b	a	c	a	b	b				
											→	a	b	a	c	a	b	b		
													→	a	b	a	c	a	b	b

Algoritmo Boyer-Moore

Observe que foram feitas **11** comparações para se encontrar o padrão na posição **14**!

Ou seja, menos comparações do que o número de caracteres até a posição em que o padrão ocorre!

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20
T:	b	a	b	b	a	b	d	a	b	b	a	a	c	a	b	a	c	a	b	b
	a	b	a	c	a	b	b													
								a	b	a	c	a	b	b						
										a	b	a	c	a	b	b				
										→	a	b	a	c	a	b	b			
											→	a	b	a	c	a	b	b		

Tabela *Delta 1*: Δ_1

O deslocamento relativo a esse critério é definido pela tabela Δ_1 .

Δ_1 tem uma entrada para cada símbolo do alfabeto.

Tabela Δ_1 :

Seja Σ um alfabeto, $P[1..m]$ um padrão e $c \in \Sigma$:

$$\Delta_1(c) = \begin{cases} m & \text{if } c \text{ não ocorre em } P \\ m - j & \text{onde } j \text{ é o máximo inteiro tal que } P[j] = c \end{cases}$$

Para o exemplo anterior:

Seja $\Sigma = \{a, b, c, d\}$ e $P = abacabb$ ($m = 7$):

i	1	2	3	4	5	6	7
P	a	b	a	c	a	b	b

$$\Delta_1(a) = 7 - 5 = 2$$

$$\Delta_1(b) = 7 - 7 = 0$$

$$\Delta_1(c) = 7 - 4 = 3$$

$$\Delta_1(char) = 7, \text{ para todo } char \notin \{a, b, c\}$$

Segundo Critério para Deslocamento

Além do critério usado para deslocamento anterior, o algoritmo também considera o deslocamento que pode ser feito considerando-se o sufixo do padrão já encontrado no texto.

Exemplo:

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18
T :	b	a	b	b	a	d	b	b	b	a	b	b	c	b	b	c	b	b
	a	b	b	c	b	b	c	b	b									

Segundo Critério para Deslocamento

Além do critério usado para deslocamento anterior, o algoritmo também considera o deslocamento que pode ser feito considerando-se o sufixo do padrão já encontrado no texto.

Exemplo:

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18
T :	b	a	b	b	a	d	b	b	b	a	b	b	c	b	b	c	b	b
	a	b	b	c	b	b	c	b	b									
							→	a	b	b	c	b	b	c	b	b		

Procura-se pela próxima ocorrência de **bb** no padrão, sendo que o caractere anterior seja diferente de **c**

Segundo Critério para Deslocamento

Além do critério usado para deslocamento anterior, o algoritmo também considera o deslocamento que pode ser feito considerando-se o sufixo do padrão já encontrado no texto.

Exemplo:

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18
T :	b	a	b	b	a	d	b	b	b	a	b	b	c	b	b	c	b	b
	a	b	b	c	b	b	c	b	b									
							→ a	b	b	a	b	b	c	b	b			

Segundo Critério para Deslocamento

Além do critério usado para deslocamento anterior, o algoritmo também considera o deslocamento que pode ser feito considerando-se o sufixo do padrão já encontrado no texto.

Exemplo:

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18
T :	b	a	b	b	a	d	b	b	b	a	b	b	c	b	b	c	b	b
	a	b	b	c	b	b	c	b	b									
							→ a	b	b	a	b	b	c	b	b			

Procura-se pela próxima ocorrência de **bbcbb** no padrão, sendo que o caractere anterior seja diferente de **c**

Segundo Critério para Deslocamento

Além do critério usado para deslocamento anterior, o algoritmo também considera o deslocamento que pode ser feito considerando-se o sufixo do padrão já encontrado no texto.

Exemplo:

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18
T :	b	a	b	b	a	d	b	b	b	a	b	b	c	b	b	c	b	b
	a	b	b	c	b	b	c	b	b									
							→ a	b	b	c	b	b	c	b	b			
									→ a	b	b	c	b	b	c	b	b	

Procura-se pela próxima ocorrência de **bbcbb** no padrão, sendo que o caractere anterior seja diferente de **c**

Segundo Critério para Deslocamento

Além do critério usado para deslocamento anterior, o algoritmo também considera o deslocamento que pode ser feito considerando-se o sufixo do padrão já encontrado no texto.

Exemplo:

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18
T :	b	a	b	b	a	d	b	b	b	a	b	b	c	b	b	c	b	b
	a	b	b	c	b	b	c	b	b									
							→ a	b	b	c	b	b	c	b	b			
									→ a	b	b	c	b	b	b	c	b	b

Tabela *Delta* 2: Δ_2

O segundo critério de deslocamento é definido na tabela Δ_2

A tabela Δ_2 possui uma entrada para cada elemento do padrão.

Tabela *Delta* 2: Δ_2

O segundo critério de deslocamento é definido na tabela Δ_2

A tabela Δ_2 possui uma entrada para cada elemento do padrão.

Considere:

- ▶ que o padrão P seja estendido com elementos de índice 0, -1, -2, ... e que essas posições terão um caractere especial \$, que não ocorre em P

Tabela *Delta* 2: Δ_2

O segundo critério de deslocamento é definido na tabela Δ_2

A tabela Δ_2 possui uma entrada para cada elemento do padrão.

Considere:

- ▶ que o padrão P seja estendido com elementos de índice 0, -1, -2, ... e que essas posições terão um caractere especial \$, que não ocorre em P
- ▶ que duas sequências $[c_1, c_2, \dots, c_k]$ e $[d_1, d_2, \dots, d_k]$ se "unificam", se:
para todo $i = 1 \dots k$: ou $c_i = d_i$ ou $c_i = \$$ ou $d_i = \$$

Tabela *Delta* 2: Δ_2

O segundo critério de deslocamento é definido na tabela Δ_2

A tabela Δ_2 possui uma entrada para cada elemento do padrão.

Considere:

- ▶ que o padrão P seja estendido com elementos de índice 0, -1, -2, ... e que essas posições terão um caractere especial \$, que não ocorre em P
- ▶ que duas sequências $[c_1, c_2, \dots, c_k]$ e $[d_1, d_2, \dots, d_k]$ se "unificam", se:
para todo $i = 1 \dots k$: ou $c_i = d_i$ ou $c_i = \$$ ou $d_i = \$$

Por exemplo, as sequências abaixo unificam-se:

Sequência 1: \$ \$ \$ \$ a b c d

Sequência 2: a b a b a b c d

Tabela *Delta* 2: Δ_2

Rightmost Plausible Reoccurrence (rpr)

Para a *substring* que se inicia na posição $j + 1$, $rpr(j)$, para $j = 1 \cdots m - 1$, é o maior k menor que ou igual a m tal que:

$$P[j + 1 \dots m] \text{ e } P[k \dots k + m - j - 1] \text{ se unificam}$$

e

$$\text{ou } k \leq 1 \text{ ou } P[k - 1] \neq P[j]$$

Tabela Δ_2 :

$$\Delta_2(j) = m + 1 - rpr(j)$$

Tabela Δ_2 : Exemplo

j	1	2	3	4	5	6	7	8	9
P	a	b	y	x	c	d	e	y	x
Δ_2	17	16	15	14	13	12	7	10	1

$\Delta_2(m)$ pode ser sempre 1

Por que $\Delta_2(7) = 7$?

$rpr(7) = 3$, pois a próxima ocorrência de yx no padrão em que o caractere anterior não é e é na posição 3.

Com isso, $\Delta_2(7) = m + 1 - rpr(7) = 9 + 1 - 3 = 7$

Mas qual é a razão? Porque o deslocamento possível, quando a comparação com o $P[7]$ der sem sucesso, corresponde a um deslocamento de 7:

j	1	2	3	4	5	6	7	8	9
P	a	b	y	x	c	d	e	y	x
						a	b	y	x
							↑		↑

deslocamento de 7

Tabela Δ_2 : Exemplo

j	1	2	3	4	5	6	7	8	9
P	a	b	y	x	c	d	e	y	x
Δ_2	17	16	15	14	13	12	7	10	1

Por que $\Delta_2(8) = 10$?

$rpr8 = 0$:

j	-1	0	1	2	3	4	5	6	7	8	9
P	\$	\$	a	b	y	x	c	d	e	y	x
		x									

\$ a b y x c d e y x
 ↑ deslocamento de 10 ↑

$$\Delta_2(8) = m + 1 - rpr(8) = 9 + 1 - 0 = 10$$

Tabela Δ_2 : Exemplo

j	1	2	3	4	5	6	7	8	9
P	a	b	y	x	c	d	e	y	x
Δ_2	17	16	15	14	13	12	7	10	1

Por que $\Delta_2(6) = 12$?

 $r_{pr6} = -2:$

j	-3	-2	-1	0	1	2	3	4	5	6	7	8	9
P	\$	\$	\$	\$	a	b	y	x	c	d	e	y	x
		e	y	x									

\$ \$ \$ a b y x c d e y x
↑ deslocamento de 12 ↑

$$\Delta_2(6) = m + 1 - rpr(8) = 9 + 1 - (-2) = 12$$

Exercício

Encontre a tabela Δ_2 para o padrão:

j	1	2	3	4	5	6	7	8	9
P	a	b	c	x	x	x	a	b	c

Resposta:

j	1	2	3	4	5	6	7	8	9
P	a	b	c	x	x	x	a	b	c
Δ_2	14	13	12	11	10	9	11	10	1

Algoritmo de Boyer-Moore

Comparam-se os caracteres a partir do último caractere do padrão em direção ao primeiro. Quando o caractere do padrão em uma posição i for diferente do caractere do texto c , calcula-se o deslocamento definido por $\Delta_1(c)$ e $\Delta_2(i)$. Soma-se o maior destes valores à posição atual do texto e recomeça o processo a partir desta posição resultante.

Busca no Texto

Exemplo: Padrão: $P = aababbaa$ e $\Sigma = \{a, b, c\}$

Tabela Δ_1 :

$$\Delta_1(a) = 8 - 8 = 0 \quad \Delta_1(b) = 8 - 6 = 2 \quad \Delta_1(c) = 8$$

Tabela Δ_2 :

j	1	2	3	4	5	6	7	8
P	a	a	b	a	b	b	a	a
Δ_2	13	12	11	10	9	8	5	1

1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21
a	a	b	a	a	b	c	a	a	a	b	a	b	a	a	b	a	b	b	a	a
a	a	b	a	b	b	a	a													

$$\Delta_1(c) = 8, \Delta_2(7) = 5: \text{maior: } 8$$

a a b a b b a a

$$\Delta_1(a) = 0, \Delta_2(5) = 9: \text{maior: } 9$$

a a b a b b a a

Apenas 14 comparações!

Complexidade

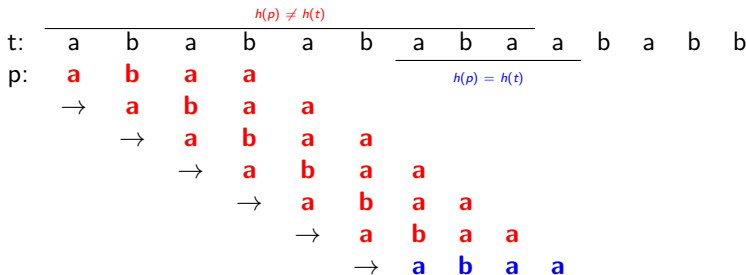
- ▶ A construção de Δ_1 pode ser feita inicializando-se um vetor do tamanho do alfabeto com o valor m e, em seguida, percorrendo o padrão linearmente. Portanto, é $O(|\Sigma| + m)$
- ▶ A construção de Δ_2 pode ser feita em $O(m)$ (D.E. Knuth, J.H. Morris, V.R. Pratt. Fast Pattern Matching in Strings. *SIAM Journal of Computing*. Vol. 6, No. 2. Junho, 1977)
- ▶ O algoritmo de busca é $O(n + m)$. O algoritmo, no entanto, tem um comportamento esperado *sublinear*, ou seja, fazem-se menos do que i comparações para se encontrar o padrão na posição i ([Boyer, Moore])

Algoritmo Rabin-Karp: ideia básica

Objetivo: comparar padrão com o texto apenas quando necessário

- Uso de função uma hashing $h(.)$ sobre o padrão e sobre cada bloco de m caracteres do texto

Se $h(p) = h(t[i \dots i + m - 1])$, verifica-se se $t[i \dots i + m - 1]$ é igual ao padrão. Caso contrário, avança-se uma posição no texto.



Construção da função hashing

Símbolos do alfabeto Σ são codificados numericamente

- ▶ Existem $|\Sigma|$ valores numéricos no alfabeto (ex.: ASC II)
- ▶ Padrão é então constituído de m dígitos numa base $\mathbf{b}$,
 $p = d_0 d_{m-2} \cdots d_{m-1}$. **Exemplo:** $\Sigma = \{0, 1, \dots, 9\}$, $\mathbf{b} = 10$
- ▶ **Com q escolhido de forma apropriada**, durante o pré-processamento do padrão,

$$h(p) = \overbrace{\left(\sum_{i=0}^{m-1} p[i] \times \mathbf{b}^{m-i-1} \right)}^{\text{pode ser muito grande}} \bmod q$$
$$h(t[0 \cdots m-1]) = \underbrace{\left(\sum_{i=0}^{m-1} t[i] \times \mathbf{b}^{m-i-1} \right)}_{\text{pode ser muito grande}} \bmod q$$

Observações

- ▶ Módulo da soma é igual ao módulo da soma dos módulos:

$$h(x) = \left(\sum_{i=0}^{m-1} x[i] \times \mathbf{b}^{m-i-1} \right) \bmod q = \left(\sum_{i=0}^{m-1} \left(x[i] \times \mathbf{b}^{m-i-1} \bmod q \right) \right) \bmod q$$

- ▶ q deve ser tal que o valor $q \times \mathbf{b}$ caiba numa palavra de computador (ex.: 64 bits)
- ▶ q deve ser preferencialmente primo para diminuir o número de colisões
- ▶ Valor de $h(x)$ é computado em $\Theta(m)$

Construindo a solução de Rabin-Karp

Primeira tentativa:

1. $v_p \leftarrow h(p)$ $\Theta(m)$
2. $v_t \leftarrow h(t[0, \dots, m-1])$ $\Theta(m)$
3. $s \leftarrow 0$
4. Enquanto $s \leq m - n$ $\Theta(n - m)$
 - 4.1 Se $v_p = v_t$
 - 4.1.1 Se $p = t[s, \dots, s + m - 1]$ $\Theta(m)$ somente se $v_p = v_t$
Retorna s
 - 4.2 $s \leftarrow s + 1$
 - 4.3 $v_t \leftarrow h(t[s, \dots, s + m - 1])$ $\Theta(m)$
5. Retorna -1

Pior caso: $\Theta(m(m - n))$ independentemente de v_p e v_t !!!

Solução: calcular v_t no passo 4.3 em $\Theta(1)$!!!

Construindo a solução de Rabin-Karp

Melhorando os cálculos para $h(t[s, \dots, s + m - 1])$

1. $v_p \leftarrow h(p)$ $\Theta(m)$
2. $v_t \leftarrow h(t[0, \dots, m - 1])$ $\Theta(m)$
3. $w \leftarrow b^{m-1} \bmod q$
4. $s \leftarrow 0$
5. Enquanto $s \leq m - n$ $\Theta(n - m)$
 - 5.1 Se $v_p = v_t$
 - 5.1.1 Se $p = t[s, \dots, s + m - 1]$
Retorna s
 - 5.2 $s \leftarrow s + 1$
 - 5.3 **Se** $s \leq m - n$
 - 5.3.1 $v_t \leftarrow (v_t - t[s - 1] \times w) \times b + t[s + m - 1]) \bmod q$ $\Theta(1)$
6. Retorna -1

Passo 5.3.1 calcula v_t usando seu valor da iteração anterior

Rabin-Karp

```
1 int rkMatcher(char t[], char p[]) {
2     long m, n, w = 1, vp = 0, vt = 0, s = 0;
3     const long q = 7919; // q primo
4
5     m = strlen(p); n = strlen(t);
6     for(int i = 1; i < m; i++) // equiv  $B^{m-1}$ 
7         w = (w * B) % q;
8     for(int i = 0; i < m; i++) { // hash inicial
9         vp = (vp * B + p[i]) % q;
10        vt = (vt * B + t[i]) % q;
11    }
12    while(s <= n - m) {
13        if (vp == vt && strncmp(p, t+s, m) == 0)
14            return s;
15        s++;
16        if (s <= n - m) {
17            vt = (B * (vt - t[s-1] * w) + t[s + m - 1]) % q;
18            if (vt < 0) vt = vt + q;
19        }
20    }
21    return -1;
22 }
```

Rabin-Karp: Complexidade

- ▶ Apenas pré-processamento: $\Theta(m)$
- ▶ Pior caso: **colisão com probabilidade 1**. Em todas as iterações, $h(p) = h(t[s, \dots, s + m - 1])$: $\Theta(m) + \Theta((n - m + 1)m) = O(nm)$
- ▶ Melhor caso: **colisão com probabilidade 0**. Com padrão não encontrado, em todas as iterações $h(p) \neq h(t[s, \dots, s + m - 1])$: $\Theta(m + n - m + 1) = \Theta(n)$
- ▶ Caso médio*: **Considere que $\Pr\{\text{Colisao}\} \approx \frac{1}{m}$** . Para cada valor de $s = 0, 1, \dots, m - n$, a verificação custa $\Theta(m)$, então, em média,

$$\Theta(m) + \sum_{s=0}^{n-m} \frac{1}{m} \Theta(m) = \Theta(m) + \Theta(n - m + 1) = \Theta(m + n)$$

* R. M. Karp e M. O. Rabin. "Efficient randomized pattern-matching algorithms". IBM Journal of Research and Development. 31 (2): 249-260.

- ▶ Casamento de padrões em texto, um problema recorrente em várias aplicações
- ▶ Força-bruta: solução simples, mas com custo maior que o necessário
- ▶ KMP: evita retrocessos no texto com base no conhecimento sobre o padrão
- ▶ Boyer-Moore: Considera alfabeto e compara o padrão da direita para esquerda. Pode haver retrocessos sobre o texto, mas com baixa probabilidade para alfabetos grandes
- ▶ Rabin-Karp: Uso de hashing. Desempenho depende da função hashing escolhida. Pode ser estendido para casamento de padrões em duas dimensões.